

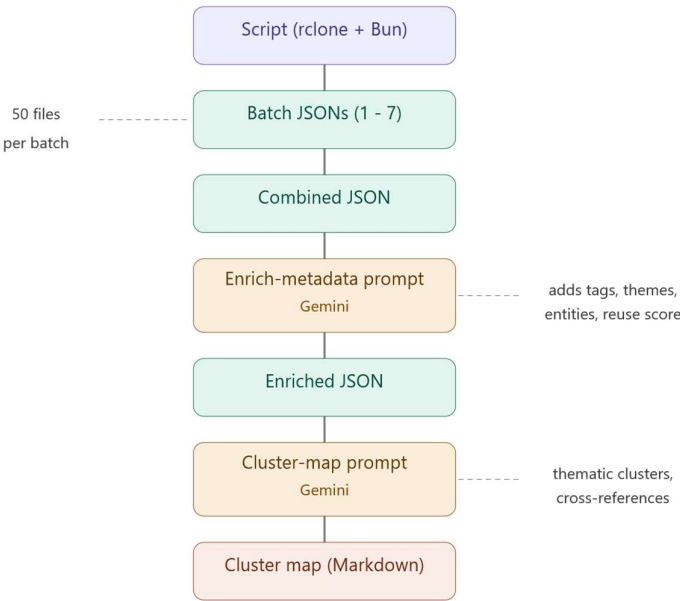
Companion Guide: Building a Metadata Layer for Your Google Drive

=====

This guide is for readers who want to run the script themselves.

It assumes no prior programming experience beyond comfort with a terminal.

WHAT YOU ARE BUILDING



A JSON file — your personal Drive index — where every document has:

- a filename and path
- a size
- a 1000-character text snippet

- (optionally) AI-enriched fields: summary, tags, themes, reuse score

Once built, this index becomes the data you feed into the AI prompts described in the main post.

BEFORE YOU BEGIN: RCLONE

rclone is a free, open-source command-line utility that connects to cloud storage including Google Drive. It is the bridge between the script and your files.

Install rclone:

Visit <https://rclone.org/install/> and follow instructions for your OS.

On Windows, download the .zip, extract, and place rclone.exe somewhere on your PATH (e.g. C:\rclone\).

Configure rclone for Google Drive:

Run: `rclone config`

Follow the interactive prompts. When asked for a name, use something short like "gdrive" or "drive" — you will reference this in the script.

Full guide: <https://rclone.org/drive/>

Verify the connection:

`rclone lsf gdrive: --max-depth 1`

If you see your Drive folders listed, rclone is working.

INSTALLING BUN

Bun is a JavaScript runtime — it runs the script.

If you already have Node.js installed, install Bun globally:

```
npm install -g bun
```

Verify:

```
bun --version
```

Alternatively, visit <https://bun.sh> for platform-specific installers.

THE SCRIPT: WHAT IT DOES

extract.js does four things:

1. Calls rclone to list all .txt files on your Drive recursively
2. Divides the list into batches of 50 (configurable)
3. Reads each file's content via rclone
4. Saves a JSON manifest: filename, path, size, and a 1000-char snippet

Each batch produces one output file:

```
drive_summary_batch_1.json
```

```
drive_summary_batch_2.json
```

... and so on.

RUNNING THE SCRIPT

1. Download extract.js from the repository <https://github.com/tustnexus/drive-tool/>
2. Open the file and update the configuration section at the top:

```
const RCLONE_EXE = "C:\\rclone\\rclone.exe"; // path to your rclone
const REMOTE_NAME = "gdrive:";           // your rclone remote name
const BATCH_SIZE = 50;                   // files per batch
const BATCH_NUM = 1;                     // current batch number
```

3. Navigate to the folder containing extract.js in your terminal:

```
cd path\to\your\folder
```

4. Run:

```
bun extract.js
```

5. When complete, increment BATCH_NUM and run again until all files are processed.

WHAT THE OUTPUT LOOKS LIKE

Each batch file is a JSON array. A single entry looks like this:

```
{  
  "fileName": "Safe Internet Group.txt",  
  "remotePath": "Notes/Safe Internet Group.txt",  
  "size": "2.39 KB",  
  "snippet": "Safe online habits of internet users are a pre-requisite  
for safe Internet adoption in India. Users — irrespective of the years  
of use — tend to be naive in their online behavior..."  
}
```

COMBINING BATCHES

After all batches are complete, combine them into one file.

In PowerShell:

```
$files = Get-Content drive_summary_batch_*.json | ConvertFrom-Json  
$files | ConvertTo-Json -Depth 10 | Set-Content drive_summary_all.json
```

Or simply paste all batch contents into a single JSON array manually into your editor.

ENRICHING WITH AI

Paste the combined JSON into enrich-metadata prompt from the main post.

The AI will append structured fields to every entry:

summary, category, tags, entities, themes, reuse score, content type.

The enriched output is what you feed into the Cluster Map prompt.

PRACTICAL TIPS

- Run one batch first to verify your rclone config is correct.
- Keep BATCH_SIZE constant across all runs.
- If a file is skipped (rclone error), the script logs a warning and continues — no data is lost.
- The snippet captures only the first 1000 characters. For most personal documents this is sufficient for AI inference.
- The script only reads files. It does not write to, move, or delete anything on your Drive.

WHAT NEXT

Once the enriched JSON is in hand, one has more than a retrieval aid. The structured fields — categories, themes, reuse scores — together form a modest but honest map of one's intellectual output over time. I am hopeful that readers will find this map useful not only for locating documents, but for understanding the recurring concerns that have shaped their work.

The cluster-map prompt is a natural next step. Submitting the enriched JSON to the cluster-map prompt will create a markdown file that surfaces thematic groupings that are not always visible when files are encountered one at a time. I would suggest revisiting this

markdown creation quarterly, as new documents will shift the clusters in ways that are often instructive. The `reuse_potential` scores, in particular, deserve attention before beginning any new project — it is worth knowing what already exists before committing effort to reconstruction.

A Drive that feels disorganised is, in most cases, simply unindexed. The material is there. This method is one way of making it findable, and I trust it will be of some practical use to those willing to spend an afternoon on the initial setup.